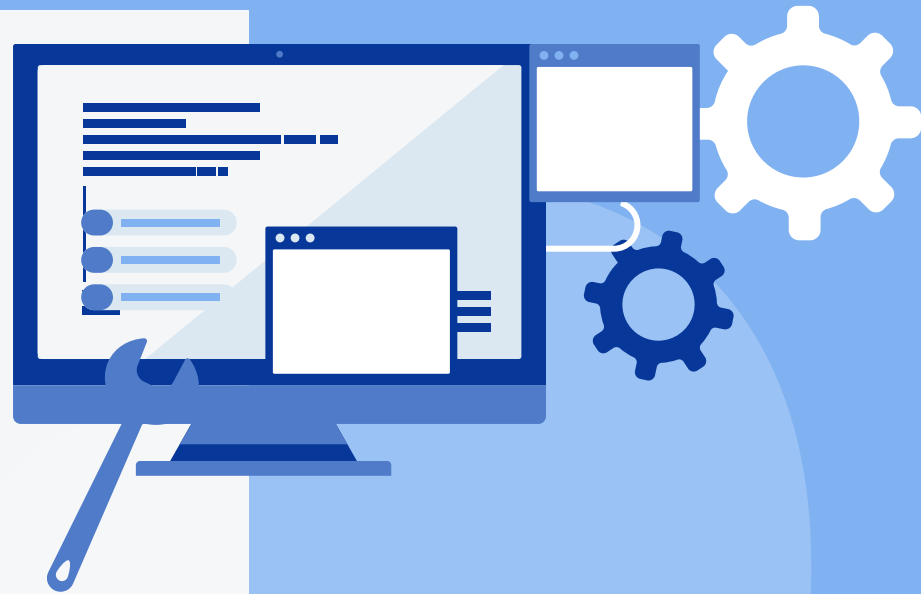


Pregled računarskog sistema i računarski softver

<https://dl.vhts.edu.rs/>

Sistemi softver za aplikacije

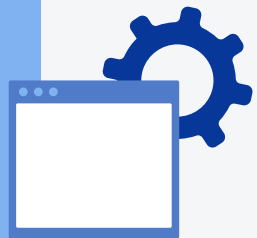
Tema 4



Autor:
saradnik Dušan Ružić

Mentor:
prof. PhD Mališa Stevanović

Četiri osnovna elementa računara su:



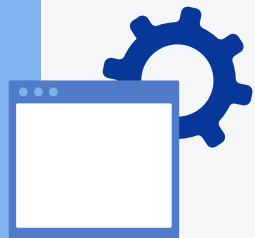
Centralna procesorska jedinica (CPU)

Glavni dio računara koji izvršava naredbe programa. Obraduje podatke, upravlja radom ostalih komponenti i izvodi aritmetičke i logičke operacije.

Radna memorija (RAM)

Privremena memorija u kojoj se čuvaju podaci i programi koji se trenutno koriste. Brza je, ali podaci se brišu kada se računar isključi.

Četiri osnovna elementa računara su:



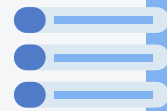
Uređaji za unos (ulazne jedinice)

Omogućavaju unos podataka i naredbi u računar. Primjeri su tastatura, miš, skener i mikrofoni.

Uređaji za izlaz (izlazne jedinice)

Prikazuju ili prenose rezultate obrade podataka korisniku. Primjeri su monitor, štampač i zvučnici.

Dve glavne kategorije registara procesora su:



Opšte namenski registri

Koriste se za privremeno čuvanje podataka i rezultata tokom izvršavanja programa. U njima se nalaze operandi (brojevi, adrese) nad kojima procesor vrši aritmetičke i logičke operacije.

Specijalni (posebni) registri

Imaju unapred određenu funkciju u radu procesora. Na primjer, čuvaju adresu naredne instrukcije, trenutno izvršavanu instrukciju ili informacije o stanju procesora (zastavice).

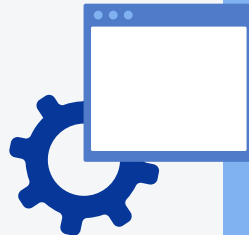
Akcije mašinskih instrukcija

Prenos podataka (Data transfer)

Premještanje podataka između registara, memorije i ulazno-izlaznih uređaja.

Aritmetičke operacije

Izvođenje matematičkih operacija nad podacima (sabiranje, oduzimanje, množenje, dijeljenje).



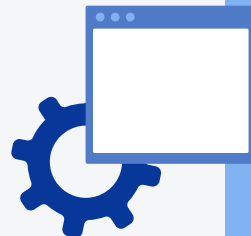
Akcije mašinskih instrukcija

Logičke operacije

Operacije nad bitovima podataka (AND, OR, NOT, XOR), poređenja i postavljanje uslova.

Kontrola toka programa (upravljanje)

Promjena redoslijeda izvršavanja instrukcija (skokovi, grananja, pozivi potprograma, povratak iz potprograma).



Prekidi (interrupts)



U računarstvu, prekid (interrupt) je signal koji privremeno zaustavlja trenutno izvršavanje programa kako bi procesor obradio neki važan događaj.

Drugim riječima, to je mehanizam kojim hardver ili softver „skreće pažnju“ procesoru da se nešto desilo i da treba odmah reagovati.



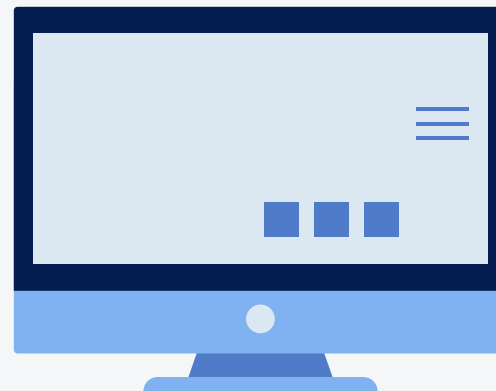
Kako funkcioniše prekid?



Procesor izvršava program (instrukcije redom).

Dogodi se neki događaj (npr. pritisak na tipku).

Šalje se signal prekida procesoru.



Kako funkcioniše prekid?



Procesor:

- pamti trenutno stanje (adresu instrukcije, registre),
- prelazi na posebnu rutinu za obradu prekida (ISR – Interrupt Service Routine).

Nakon obrade, procesor se vraća na prethodni program i nastavlja rad.



Vrste prekida



1. Hardverski prekidi



Dolaze od spoljašnjih uređaja:

- tastatura
- miš
- disk
- mrežna kartica
- tajmer



Primer: kada pritisneš tipku, tastatura šalje signal procesoru.

2. Softverski prekidi



Generišu ih programi putem posebnih instrukcija.

Primer:

- sistemski pozivi (traženje usluge od operativnog sistema)
- dijeljenje nulom (greška u programu)



Zašto su prekidi važni?



Bez prekida, procesor bi morao stalno provjeravati (polling) da li se nešto desilo.

Prekidi omogućavaju:

- efikasniji rad procesora
- bržu reakciju na događaje
- multitasking (više programa istovremeno)
- komunikaciju sa hardverom



Analogija, primer



Jednostavna analogija

- Zamisli da učiš (**program se izvršava**).
- Telefon zazvoni (**prekid**).
- Prekineš učenje, javiš se (**obrada prekida**), pa se vratiš učenju tamo gdje si stao.



Obrada višestrukih prekida



Obrada višestrukih prekida znači situaciju kada se dok se jedan prekid već obrađuje pojavi novi prekid.

Postoji nekoliko načina kako **sistem to rešava:**





Red čekanja (sekvencijalna obrada)

Ako tokom obrade jednog prekida stigne drugi:

- novi prekid se pamti (stavlja se u čekanje),
- procesor prvo završi trenutnu rutinu,
- zatim obrađuje sljedeći prekid.

Prekidi se rešavaju jedan po jedan.



Prioriteti prekida



Svaki prekid ima svoj nivo prioriteta.

Ako tokom obrade jednog prekida stigne drugi:

- ako ima **niži prioritet** → **čeka**
- ako ima **viši prioritet** → **prekida trenutnu obradu**

To se naziva **ugnježđeni prekid (nested interrupt)**.



Prioriteti prekida



Primer:

- Završetak štampanja (nizak prioritet)
- Signal kvara sistema (visok prioritet)

Sistem će prvo obraditi hitniji događaj.



Maskiranje prekida



Procesor može privremeno onemogućiti (**maskirati**) neke prekide dok se obrađuje važan zadatak.

To znači:

- određeni prekidi se ignorišu dok traje obrada
- nakon završetka, prekidi se ponovo dozvoljavaju



Kako to tehnički izgleda?



Kada dođe novi prekid:

- Procesor čuva trenutno stanje (registre, adresu instrukcije).
- Proverava prioritet.
- Po potrebi prekida trenutnu rutinu.
- Nakon obrade, vraća se tačno tamo gdje je stao.



Jednostavna analogija, primer



Zamisli da razgovaraš telefonom:

Ako te neko zove na drugi telefon (manje važno) → ignorišeš dok ne završiš.

Ako zazvoni alarm za požar → odmah prekidaš razgovor i reaguješ.



Prekidi - rezime



Ukratko:

Višestruki prekidi se obrađuju pomoću reda čekanja, prioriteta i maskiranja, kako bi sistem uvijek prvo reagovao na najvažnije događaje.



Hijerarhija memorije



Elementi u hijerarhiji memorije (registri, keš, RAM, disk...) razlikuju se prema nekoliko osnovnih kriterijuma:



1. Brzina pristupa (vreme pristupa)



Koliko brzo procesor može doći do podataka.

- **Registri** → najbrži
- **Cache** → vrlo brz
- **RAM** → sporiji
- **Disk/SSD** → najsporiji

Što je memorija bliža procesoru, to je brža.



2. Kapacitet (veličina)



Koliko podataka može da pohrani.

- **Registri** → **veoma mali kapacitet**
- **Cache** → **mali**
- **RAM** → **veći**
- **Disk** → **ogroman kapacitet**

Što je memorija veća, obično je sporija.





3. Cena po bitu

Koliko košta pohrana jedne jedinice podataka.

- **Registri** → najskuplji
- **Cache** → skup
- **RAM** → jeftiniji
- **Disk** → najjeftiniji

Brža memorija je skuplja.





4. Učestalost korišćenja

Koliko se često podaci koriste.

- **Registri** → trenutno aktivni podaci
- **Cache** → često korišćeni podaci
- **RAM** → aktivni programi
- **Disk** → retko korišćeni podaci

Hijerarhija se zasniva na principu lokalnosti (češće korišćeni podaci drže se bliže procesoru).



5. Trajnost podataka



Da li podaci ostaju sačuvani nakon gašenja računara.

- **Registri, cache, RAM** → nestalna memorija (volatile)
- **Disk, SSD** → stalna memorija (non-volatile)



Ukratko



Kako se spuštamo niz hijerarhiju memorije:

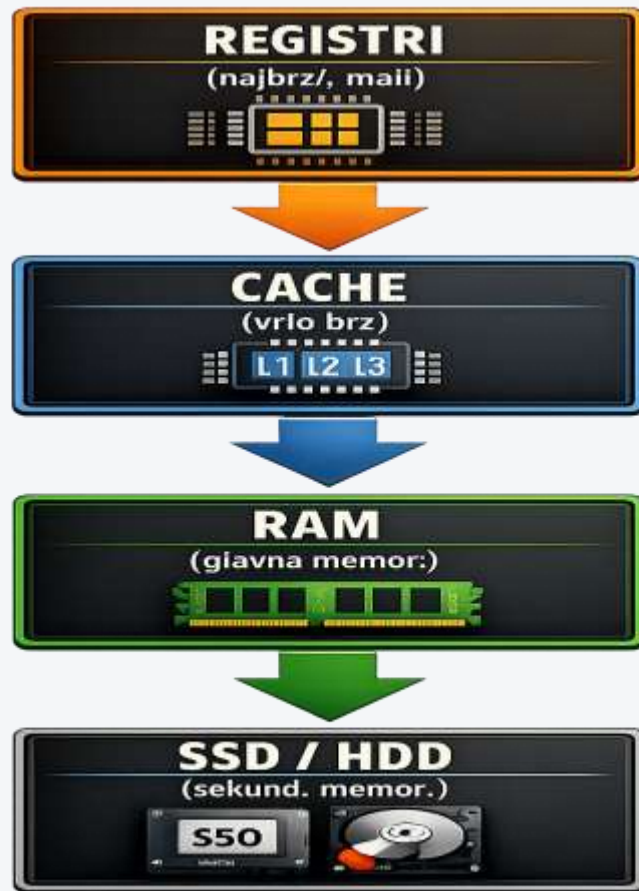
- brzina ↓
- cena po bitu ↓
- kapacitet ↑
- trajnost ↑



Pravila koja važe:

Kako ideš od vrha ka dnu:

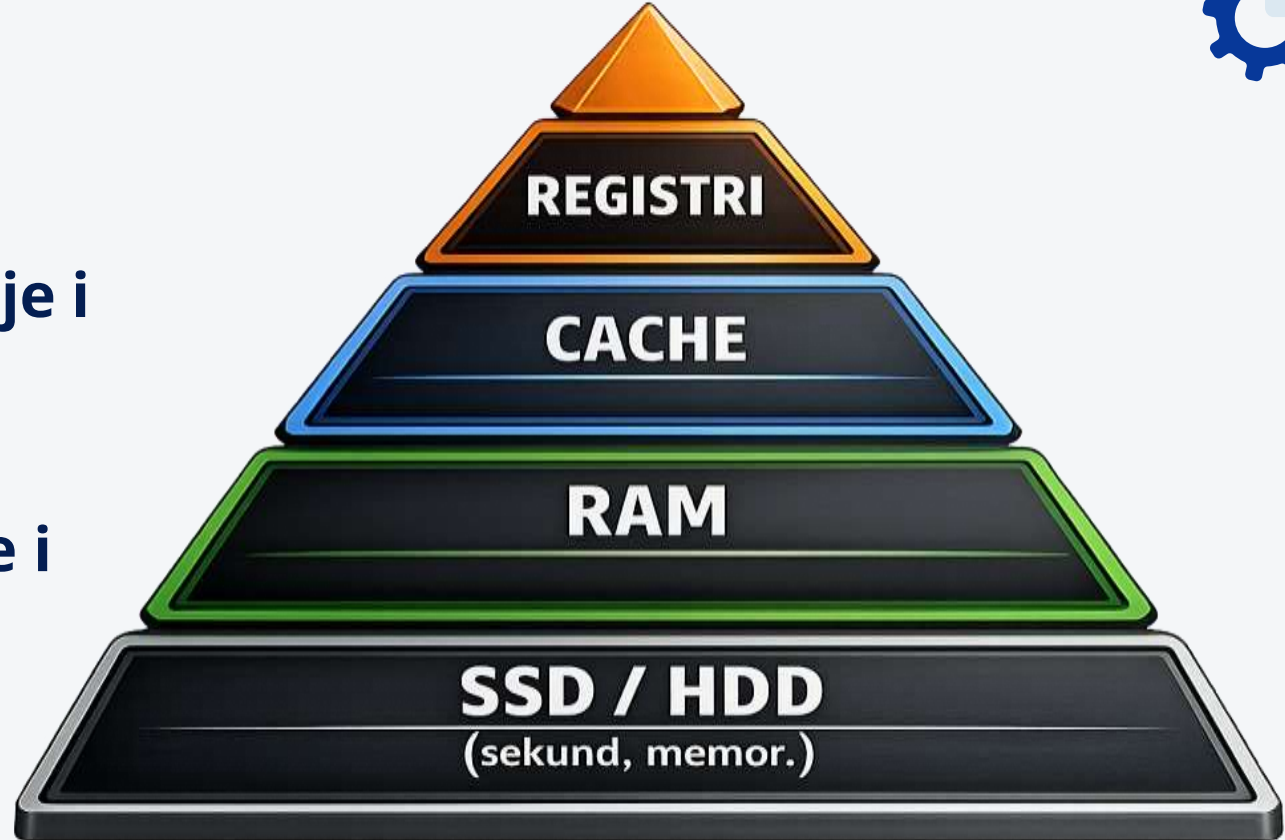
- brzina opada
- cena po bitu opada
- kapacitet raste
- trajnost podataka raste





**Vrh = najmanje i
najbrže**

**Dno = najveće i
najsporije**



Šta je keš memorija (najjednostavnije)?



Keš (cache) memorija je vrlo brza, mala memorija koja se nalazi između procesora i RAM-a.

Njena uloga je da čuva podatke koje procesor često koristi, kako bi ih dobio mnogo brže nego iz RAM-a.



Šta je keš memorija (najjednostavnije)?



Možeš je zamisliti kao:

- **Radni sto (keš)**
- **Ormar (RAM)**

Stvari koje ti stalno trebaju držiš na stolu, jer ti je brže nego stalno ustajati do ormara.



Zašto je bitna?

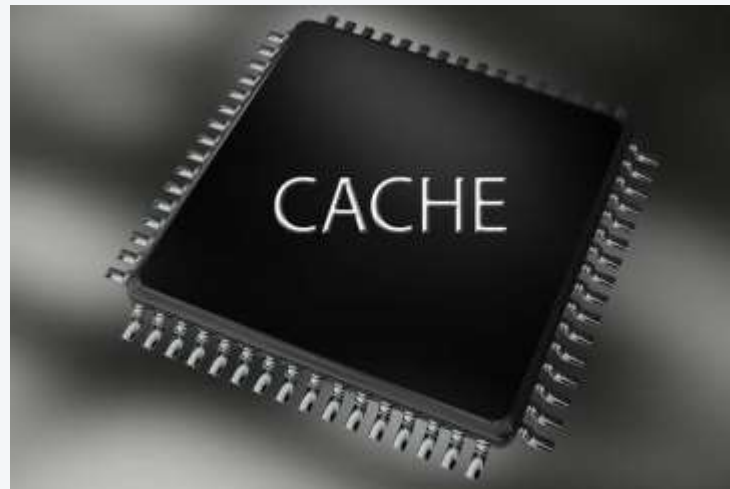


Procesor je mnogo brži od RAM memorije.

Ako bi morao svaki put čekati RAM, sistem bi bio spor.

Keš memorija:

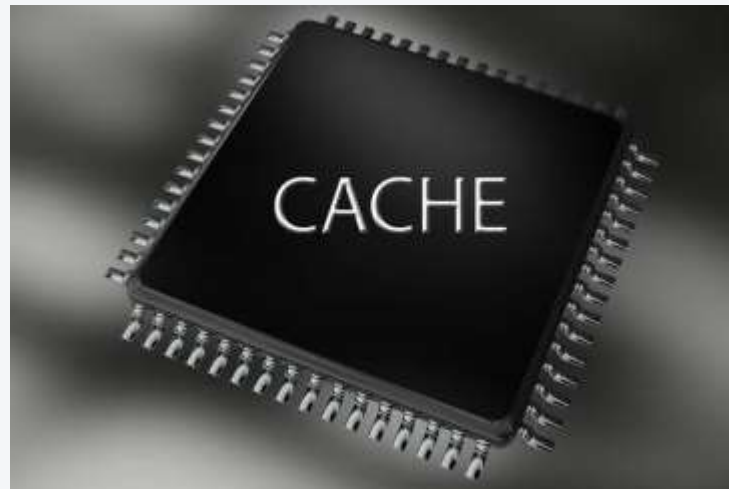
- smanjuje čekanje procesora
- ubrzava izvršavanje programa
- povećava ukupne performanse računara



Njena glavna uloga



- ✓ Čuva najčešće korišćene podatke
- ✓ Smanjuje broj pristupa sporijoj memoriji
- ✓ Automatski radi (korisnik je ne vidi direktno)

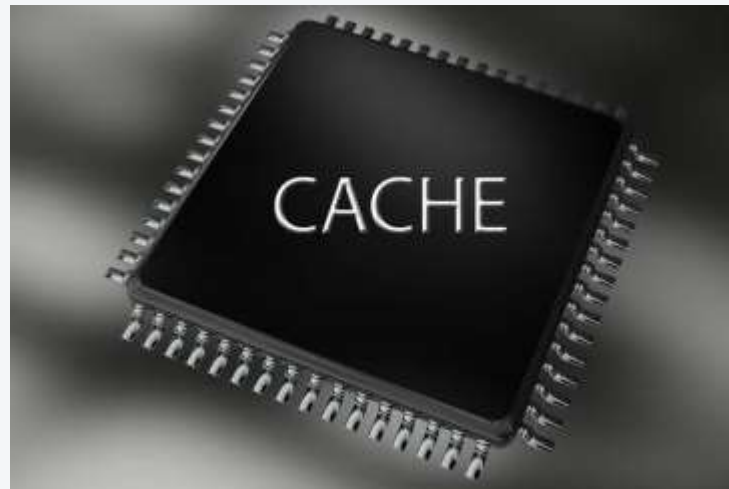


Njena glavna uloga



**Procesor prvo traži
podatak u:**

- Registrima
- Kešu
- RAM-u
- Disku



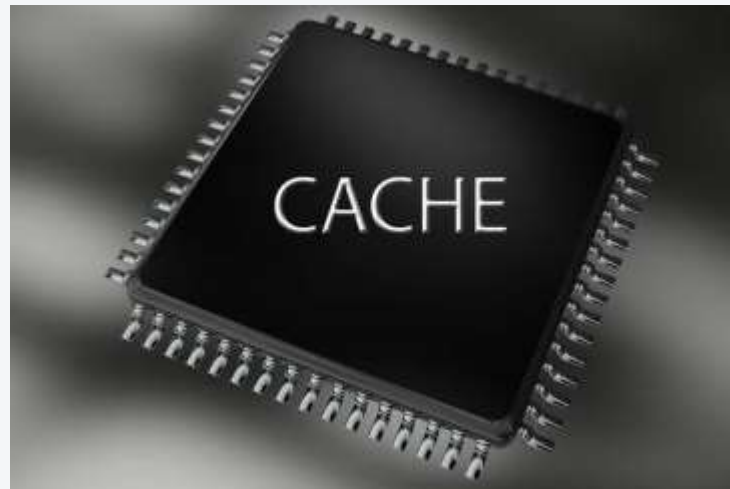
Ukratko



Što je memorija:

- **bliža procesoru** → brža je → ali manja
- **dalje od procesora** → sporija je → ali veća

Keš se nalazi između registara i RAM-a — i zato predstavlja kompromis: mnogo brža od RAM-a, ali mnogo manja po veličini.



L1, L2 i L3 keš – kao radni prostor

Zamisli da procesor radi za stolom.

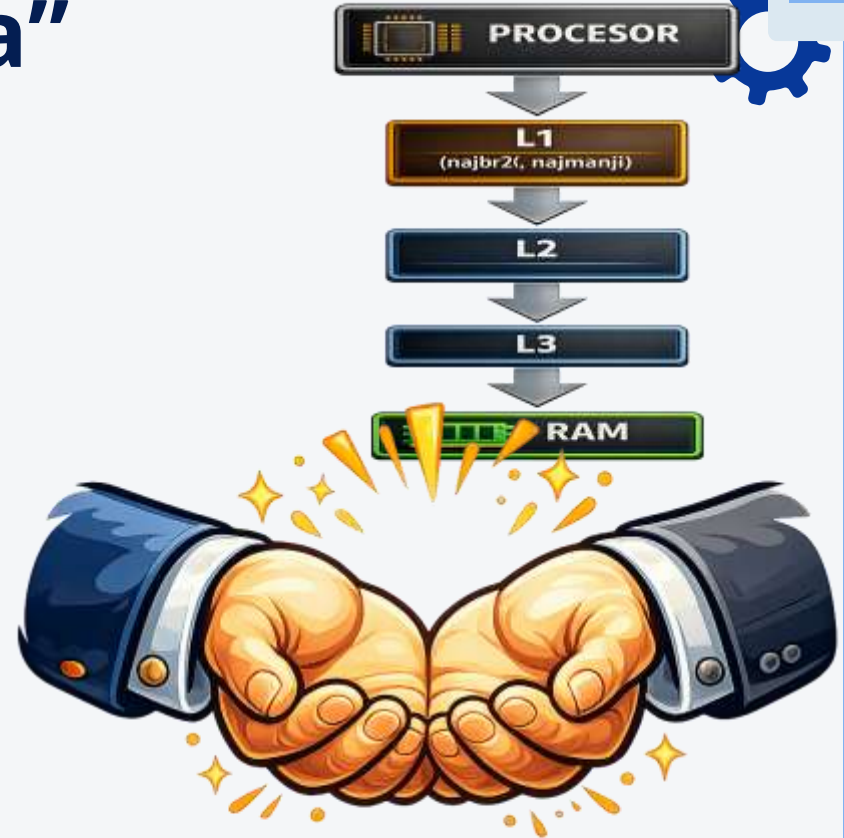


L1 keš – „u rukama”

- Najbliži procesoru
- Najbrži
- Najmanji

Svako jezgro procesora ima svoj L1

To su podaci koje procesor koristi upravo sada.



L2 keš – „na stolu”



- Malo sporiji od L1
- Malo veći
- Obično takođe poseban za svako jezgro
- Tu su podaci koji će vrlo vjerovatno uskoro trebati.



L3 keš – „u fioci stola”



- Sporiji od L1 i L2
- Znatno veći
- Najčešće zajednički za sva jezgra procesora
- Tu se drže podaci koji su često korišćeni, ali ne baš stalno.



Kako to funkcioniše u praksi?



Kad procesoru treba podatak:

- **Provjeri L1 → ako je tu, odmah ga koristi**
- **Ako nije, provjeri L2**
- **Ako nije, provjeri L3**

Ako nije, ide u RAM (sporije)

Ako ga nađe u RAM-u, kopira ga nazad u keš da bi sljedeći put bilo brže.



Ukratko



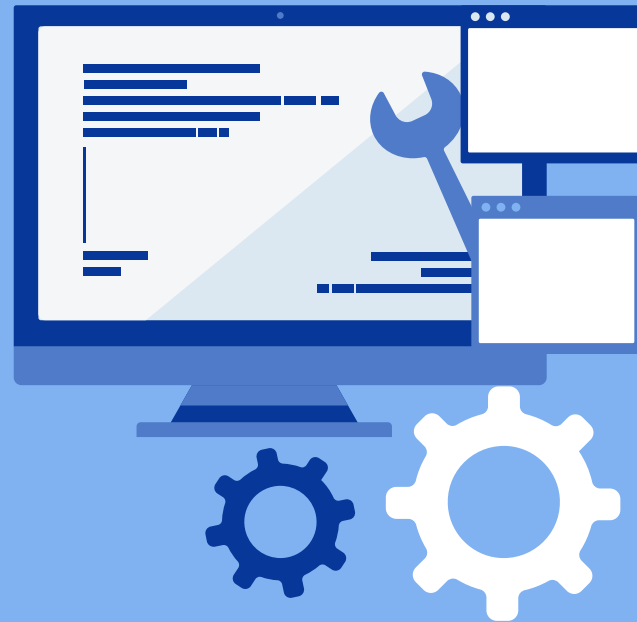
- **L1 = najbrži, najmanji, najbliži**
- **L2 = sredina**
- **L3 = najveći keš, ali sporiji**

Svi zajedno služe da procesor što rjeđe mora čekati RAM



**Hvala na
pažnji.**

**Pitanja su
dobrodošla.**



**d96rsd.com
dr@d96rsd.com
d96rsd@gmail.com**